

Understanding Recursive Linear Interpolation and Equation (12)

To understand this passage and Equation (12), let's first break down the terminology, inspect the equation, and then walk step by step through a concrete numerical toy example.

1. What Is Meant by 0th-Order, 1st-Order, and 2nd-Order Models?

In language modeling, the **order** (n) corresponds to the length of the n -gram unit, or how many previous words of context are used to predict the next word.

- **2nd-Order Model ($n = 2$, Bigram Model):** Predicts the current word w_i based on **one preceding word** (w_{i-1}).
 - **Formula:** $p(w_i \mid w_{i-1})$
 - **1st-Order Model ($n = 1$, Unigram Model):** Predicts the current word w_i based on **zero preceding words**. It ignores history and uses relative word frequencies across the corpus.
 - **Formula:** $p(w_i)$
 - **0th-Order Model ($n = 0$, Uniform Model):** Ignores both history and word frequencies, assuming every word in the vocabulary V is equally likely.
 - **Formula:** $p_{\text{unif}}(w_i) = \frac{1}{|V|}$
-

2. Deconstructing Equation (12)

Equation (12) in the paper defines **recursive linear interpolation**, specifically within the Jelinek-Mercer framework:

$$p_{\text{interp}}(w_i \mid w_{i-n+1}^{i-1}) = \lambda_{w_{i-n+1}^{i-1}} p_{\text{ML}}(w_i \mid w_{i-n+1}^{i-1}) + (1 - \lambda_{w_{i-n+1}^{i-1}}) p_{\text{interp}}(w_i \mid w_{i-n+2}^{i-1})$$

Where:

- w_{i-n+1}^{i-1} is shorthand for the history sequence of $n - 1$ words.
- p_{ML} is the Maximum Likelihood (raw-count) probability estimate.

- λ , a number between 0 and 1, is the interpolation weight assigned to the higher-order Maximum Likelihood model.
 - $1 - \lambda$ is the remaining probability mass handed down to the $(n - 1)$ -th-order smoothed model.
 - **Recursion:** If $n = 2$ (bigram), the model blends the bigram ML model with the 1st-order (unigram) smoothed model. To calculate the 1st-order smoothed model, it blends the unigram ML model with the 0th-order (uniform) model.
-

3. Step-by-Step Toy Example

Setup

- **Vocabulary:**

$$V = \{\text{"cat"}, \text{"dog"}, \text{"sat"}\}$$

Therefore, $|V| = 3$.

- **Training corpus** containing four total words:
 - "cat sat"
 - "dog sat"

Raw Counts

- **Unigram counts:**

$$c(\text{"cat"}) = 1, \quad c(\text{"dog"}) = 1, \quad c(\text{"sat"}) = 2$$

Total number of words:

$$N = 4$$

- **Bigram counts:**

$$c(\text{"cat sat"}) = 1, \quad c(\text{"cat dog"}) = 0$$

Goal

Calculate the smoothed 2nd-order probabilities for two test bigrams:

1. $p_{\text{interp}}(\text{"sat"} \mid \text{"cat"})$, which was seen in training.
2. $p_{\text{interp}}(\text{"dog"} \mid \text{"cat"})$, which was not seen in training.

Assume fixed interpolation weights for simplicity:

$$\lambda_2 = 0.6, \quad \lambda_1 = 0.7$$

Step 0: 0th-Order Model ($n = 0$)

The uniform probability for any word in the vocabulary V is:

$$p_{\text{smooth},0}(w) = \frac{1}{|V|} = \frac{1}{3} \approx 0.333$$

Step 1: 1st-Order Model ($n = 1$, Unigram)

First, calculate the Maximum Likelihood unigram probabilities:

$$p_{\text{ML}}(\text{"sat"}) = \frac{c(\text{"sat"})}{N} = \frac{2}{4} = 0.50$$
$$p_{\text{ML}}(\text{"dog"}) = \frac{c(\text{"dog"})}{N} = \frac{1}{4} = 0.25$$

Now apply Equation (12) at $n = 1$, interpolating the unigram ML model with the 0th-order uniform model.

For "sat"

$$\begin{aligned} p_{\text{interp},1}(\text{"sat"}) &= \lambda_1 \cdot p_{\text{ML}}(\text{"sat"}) + (1 - \lambda_1) \cdot p_{\text{smooth},0}(\text{"sat"}) \\ &= (0.7 \times 0.50) + (0.3 \times 0.333) \\ &= 0.350 + 0.100 \\ &= \mathbf{0.450} \end{aligned}$$

For "dog"

$$\begin{aligned} p_{\text{interp},1}(\text{"dog"}) &= \lambda_1 \cdot p_{\text{ML}}(\text{"dog"}) + (1 - \lambda_1) \cdot p_{\text{smooth},0}(\text{"dog"}) \\ &= (0.7 \times 0.25) + (0.3 \times 0.333) \\ &= 0.175 + 0.100 \\ &= \mathbf{0.275} \end{aligned}$$

Step 2: 2nd-Order Model ($n = 2$, Bigram)

Now calculate the Maximum Likelihood bigram probabilities:

$$p_{\text{ML}}(\text{"sat"} \mid \text{"cat"}) = \frac{c(\text{"cat sat"})}{c(\text{"cat"})} = \frac{1}{1} = 1.00$$

$$p_{\text{ML}}(\text{"dog"} \mid \text{"cat"}) = \frac{c(\text{"cat dog"})}{c(\text{"cat"})} = \frac{0}{1} = 0.00$$

Finally, apply Equation (12) at $n = 2$, interpolating the bigram ML model with the 1st-order smoothed model.

1. Observed Bigram: "cat sat"

$$\begin{aligned} p_{\text{interp},2}(\text{"sat"} \mid \text{"cat"}) &= \lambda_2 \cdot p_{\text{ML}}(\text{"sat"} \mid \text{"cat"}) + (1 - \lambda_2) \cdot p_{\text{interp},1}(\text{"sat"}) \\ &= (0.6 \times 1.00) + (0.4 \times 0.450) \\ &= 0.600 + 0.180 \\ &= \mathbf{0.780} \end{aligned}$$

2. Unseen Bigram: "cat dog"

$$\begin{aligned} p_{\text{interp},2}(\text{"dog"} \mid \text{"cat"}) &= \lambda_2 \cdot p_{\text{ML}}(\text{"dog"} \mid \text{"cat"}) + (1 - \lambda_2) \cdot p_{\text{interp},1}(\text{"dog"}) \\ &= (0.6 \times 0.00) + (0.4 \times 0.275) \\ &= 0.000 + 0.110 \\ &= \mathbf{0.110} \end{aligned}$$

Why This Matters

Notice that even though "cat dog" was **never seen** in the training corpus, meaning

$$p_{\text{ML}}(\text{"dog"} \mid \text{"cat"}) = 0$$

recursive linear interpolation prevents the **zero-probability problem** by handing probability mass down to the unigram and uniform estimates.

As a result, the unseen bigram receives a non-zero probability:

$$\boxed{0.110}$$

This is one of the key advantages of interpolation-based smoothing in language models.

1. Have We Discussed Backoff Yet?

No, we haven't formally covered backoff yet. We only looked at **recursive linear interpolation** (Jelinek-Mercer style) in the previous calculation.

To keep the distinction crystal clear for **Pass 2**:

- **Interpolation: Always** mixes predictions from both the higher-order and lower-order models. Even if you have seen the bigram 1,000 times, the unigram model still gets a small vote.
- **Backoff: Only** uses the lower-order model when the higher-order count is zero or below a threshold. If the higher-order n-gram was seen, backoff uses only the discounted higher-order probability.

2. Why $p_{\text{interp},1}(\text{"sat"})$ and Not $p_{\text{interp},1}(\text{"cat"})$?

To never forget this, keep one rule in mind:

Always predict the target word, never the history.

The Core Concept

In language modeling, every conditional probability has two distinct roles:

$$\underbrace{P(\text{Target} \mid \text{History})}_{\text{Predicting } w_i \text{ given } w_{i-1}}$$

For the bigram "cat sat" :

- $w_{i-1} = \text{"cat"}$ is the **history** — what already happened.
- $w_i = \text{"sat"}$ is the **target** — the next word you are trying to predict.

What Happens During Interpolation or Backoff?

When you drop from a higher-order model (bigram) to a lower-order model (unigram), **you throw away the history, not the target.**

1. Bigram Model ($n = 2$)

The bigram model asks:

*Given that the previous word was **"cat"**, what is the probability that the next word is **"sat"**?*

$$P_{\text{bigram}}(\text{"sat"} \mid \text{"cat"})$$

2. Unigram Model ($n = 1$)

The unigram model asks:

*If I ignore the previous word entirely, what is the probability that the current word being generated is "**sat**" in general?*

$$P_{\text{unigram}}(\text{"sat"})$$

Why Calculating $P(\text{"cat"})$ Would Be Wrong

If you calculated:

$$P(\text{"cat"})$$

at the unigram level, you would be asking:

What is the probability of the word "cat" appearing?

But you are not trying to predict "cat" .

"cat" has already been spoken or written. You are trying to predict **what word comes after "cat"** .

Notice how, in Equation (12) of the paper, the variable w_i , representing the **target word**, stays identical on both sides of the equation:

$$p_{\text{interp}}(w_i | w_{i-1}) = \lambda \cdot p_{\text{ML}}(w_i | w_{i-1}) + (1 - \lambda) \cdot p_{\text{interp}}(w_i)$$

The history w_{i-1} simply disappears in the lower-order term, leaving w_i , or "sat" , as the word being evaluated.